

BABES-BOLYAI UNIVERSITY  
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE  
MASTER'S PROGRAMME IN HIGH PERFORMANCE COMPUTING AND  
BIG DATA ANALYTICS

---

# The Evolution of Automated Fake News Detection

---

Research Report

**Author:**

Tudor Cristian Murariu  
Master's Student, Year II

**Scientific Coordinator:**

Lect. Dr. Alina-Delia Călin  
Faculty of Mathematics and  
Computer Science

Cluj-Napoca, 2026

## Abstract

The proliferation of misinformation across digital platforms has made automated fake news detection one of the most urgent and actively studied problems in natural language processing. This report surveys the full spectrum of computational approaches to the problem, from traditional machine-learning classifiers built on hand-crafted features to sequence models grounded in deep learning and, finally, to large pre-trained Transformers such as BERT. All methods are evaluated experimentally on the ISOT Fake News Dataset, a publicly available corpus of approximately 45 000 labelled news articles. The practical component of this work covers three families of models: classical methods (Multinomial Naive Bayes, Logistic Regression, and Random Forest operating on TF-IDF representations), recurrent architectures (a single-layer RNN and a bidirectional LSTM implemented in PyTorch), and a BERT-based fine-tuning approach. Results show a clear progression in detection performance as model complexity grows, with the bidirectional LSTM achieving near-perfect accuracy (99.04%) and BERT fine-tuning reaching **99.87%**, the highest result across all models. The report situates each model within its historical and theoretical context, offering a unified view of how the field has evolved over the past decade.

## Contents

|          |                                                          |          |
|----------|----------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                      | <b>3</b> |
| <b>2</b> | <b>Background and Related Work</b>                       | <b>4</b> |
| 2.1      | Classical Machine Learning . . . . .                     | 4        |
| 2.2      | Deep Learning Approaches . . . . .                       | 4        |
| 2.3      | Transformer-Based and State-of-the-Art Methods . . . . . | 5        |
| <b>3</b> | <b>Theoretical Framework</b>                             | <b>5</b> |
| 3.1      | Text Preprocessing . . . . .                             | 5        |
| 3.2      | TF-IDF Representation . . . . .                          | 5        |
| 3.3      | Recurrent Neural Networks . . . . .                      | 6        |
| 3.4      | Long Short-Term Memory . . . . .                         | 6        |
| 3.5      | Bidirectional LSTM . . . . .                             | 7        |
| 3.6      | Attention Mechanism and Transformers . . . . .           | 7        |
| 3.7      | BERT . . . . .                                           | 7        |
| <b>4</b> | <b>Dataset Analysis</b>                                  | <b>8</b> |
| 4.1      | The ISOT Fake News Dataset . . . . .                     | 8        |
| 4.2      | Preprocessing . . . . .                                  | 8        |
| 4.3      | Train / Validation / Test Split . . . . .                | 9        |

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| <b>5</b> | <b>Classical Machine Learning Experiments</b> | <b>9</b>  |
| 5.1      | Methodology . . . . .                         | 9         |
| 5.2      | Results . . . . .                             | 10        |
| <b>6</b> | <b>Recurrent Neural Network Experiments</b>   | <b>11</b> |
| 6.1      | Experimental Setup . . . . .                  | 11        |
| 6.2      | Vanilla RNN . . . . .                         | 12        |
| 6.2.1    | Architecture . . . . .                        | 12        |
| 6.2.2    | Training Curves and Results . . . . .         | 12        |
| 6.3      | Bidirectional LSTM . . . . .                  | 13        |
| 6.3.1    | Architecture . . . . .                        | 13        |
| 6.3.2    | Training Curves and Results . . . . .         | 14        |
| <b>7</b> | <b>BERT-Based Classification</b>              | <b>15</b> |
| 7.1      | Approach . . . . .                            | 15        |
| 7.2      | Results . . . . .                             | 15        |
| <b>8</b> | <b>Results and Discussion</b>                 | <b>17</b> |
| 8.1      | Comparative Summary . . . . .                 | 17        |
| 8.2      | Key Observations . . . . .                    | 18        |
| <b>9</b> | <b>Conclusions</b>                            | <b>18</b> |

# 1 Introduction

Few phenomena of the digital age have attracted as much alarm, from government bodies, journalists, and researchers alike, as the rapid spread of fabricated or deliberately misleading news content. The term “fake news” entered mainstream discourse with particular force during the 2016 US presidential election, when it became clear that false stories were being shared millions of times across social platforms, often outpacing corrections by reputable outlets [1]. The consequences extend well beyond partisan politics: misinformation about vaccines, natural disasters, and public health crises has demonstrably impaired decision-making at both individual and institutional levels.

Manual fact-checking, while indispensable, cannot scale to the volume of content that modern social media platforms generate. A single major platform may process hundreds of millions of posts per day, making human review a last resort rather than a first line of defence. This reality has motivated a substantial body of research into *automated* detection systems that can operate in real time and at scale.

The intellectual history of the field reflects broader shifts in machine learning. The earliest computational approaches to deception detection relied on carefully engineered features such as word counts, punctuation density, and readability scores [2]. These methods were tractable and interpretable, but their accuracy was limited by the expressiveness of the features one could design by hand. As labelled corpora grew and neural architectures matured, researchers turned to recurrent networks capable of modelling the sequential structure of text, and subsequently to Transformer-based models that could leverage vast stores of unlabelled data through self-supervised pre-training [3].

This report documents a systematic attempt to trace and reproduce that evolution. Starting from a classical feature-engineering pipeline, we progressively introduce more powerful architectures and measure how each step changes detection accuracy on a shared benchmark dataset. The work is grounded in both theoretical exposition and hands-on implementation in Python (scikit-learn and PyTorch), and is intended as a self-contained reference that connects the theoretical underpinnings of each approach with empirically observed behaviour.

The remainder of the report is structured as follows. Section 2 reviews the relevant literature. Section 3 provides the theoretical background for the models used. Section 4 describes the dataset and preprocessing pipeline. Sections 5–7 present the three experimental phases. Section 8 synthesises the results, and Section 9 concludes.

## 2 Background and Related Work

The study of fake news detection sits at the intersection of natural language processing, computational social science, and information security. Wardle & Derakhshan [4] provided an influential conceptual framework that distinguishes between *mis-information* (false content shared without malicious intent), *dis-information* (false content deliberately created to deceive), and *mal-information* (true content weaponised for harm). For the purposes of automated classification, the distinction often collapses to a binary label, but the broader taxonomy is useful for understanding the limits of text-only detectors, a satirical piece might be factually false yet carry no malicious intent.

### 2.1 Classical Machine Learning

The first wave of automated detection systems treated fake news classification as a standard text categorisation problem. Conroy et al. [2] surveyed linguistic approaches and identified reliable cues including hedging language, first-person pronouns, and irregular punctuation. Ahmed et al. [5] demonstrated that character-level and word-level n-gram features fed to a linear classifier could achieve accuracy above 90% on a corpus scraped from satirical sites and mainstream news agencies. These methods benefit from simplicity and interpretability, but their reliance on surface-level statistics makes them vulnerable to adversarial content that mimics the stylistic conventions of reputable journalism.

### 2.2 Deep Learning Approaches

Recurrent neural networks brought a qualitative shift by allowing models to capture the sequential, long-range dependencies that characterise coherent (or incoherent) narrative structure. Bahad et al. [6] showed that a bidirectional LSTM significantly outperforms bag-of-words baselines on multiple fake news benchmarks, attributing the gain to the model’s ability to integrate information from both past and future context. Ruchansky et al. [7] went further with a hybrid architecture (CSI) that jointly models article text, user responses, and temporal propagation signals, recognising that linguistic content alone is insufficient for detecting sophisticated disinformation campaigns.

The development of word embeddings (Word2Vec, GloVe) and, later, contextualised representations was critical for these deep approaches. Rather than representing words as sparse one-hot vectors, dense embeddings place semantically similar words in nearby regions of a continuous space, allowing models to generalise across paraphrases and synonyms.

## 2.3 Transformer-Based and State-of-the-Art Methods

The landmark paper by Vaswani et al. [8] introduced the self-attention mechanism and the Transformer architecture, fundamentally changing the trajectory of NLP research. BERT [3], pre-trained on 3.3 billion words with a masked-language-modelling objective, established new state-of-the-art results across virtually every NLP benchmark when fine-tuned with a single task-specific layer. Its application to fake news detection was nearly immediate: the contextualised token representations it produces are far richer than those of RNN-based models because every token attends to every other token simultaneously.

More recently, hybrid architectures that combine Transformer encoders with graph neural networks have been proposed to model propagation dynamics alongside content [9]. Large language models are also beginning to be used not just as classifiers but as reasoning engines that can generate and evaluate evidence chains for a given claim [10]. Zhou & Zafarani [11] provide a comprehensive survey of the field up to 2021, while Kumar et al. [12] offer a more recent comparison of NLP-based classifiers across multiple benchmarks.

## 3 Theoretical Framework

### 3.1 Text Preprocessing

Before any machine-learning model can process text, raw strings must be converted into a numerical representation. The preprocessing pipeline used in this work consists of three steps: *lowercasing* (eliminating case-based duplicates), *URL and punctuation removal* (stripping noise that carries no semantic signal), and *whitespace normalisation*. For the classical models, tokenisation was handled by NLTK’s `word_tokenize` function, followed by stopwords removal. For the deep models, custom tokenisation was used to build a fixed vocabulary of the most frequent terms.

### 3.2 TF-IDF Representation

Term Frequency–Inverse Document Frequency (TF-IDF) is a classical weighting scheme that transforms a document into a sparse vector whose coordinates correspond to vocabulary terms. For term  $t$  in document  $d$  and a corpus  $D$ :

$$\text{TF-IDF}(t, d, D) = \underbrace{\frac{f_{t,d}}{\sum_{t'} f_{t',d}}}_{\text{TF}} \times \underbrace{\log\left(\frac{|D|}{|\{d' \in D : t \in d'\}|}\right)}_{\text{IDF}} \quad (1)$$

High TF-IDF scores highlight terms that appear frequently within a particular document but rarely across the corpus, precisely the kind of distinctive vocabulary that can differentiate styles of reporting. The scikit-learn implementation used here caps the vocabulary at 5 000 features, keeping memory and training time manageable.

### 3.3 Recurrent Neural Networks

A Recurrent Neural Network (RNN) processes a sequence of tokens  $x_1, x_2, \dots, x_T$  one step at a time, maintaining a hidden state  $h_t$  that summarises all tokens seen so far:

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t + b_h) \quad (2)$$

After the final token  $x_T$ , the hidden state  $h_T$  serves as a fixed-size representation of the entire input sequence, which is passed to a linear classifier. While conceptually elegant, vanilla RNNs suffer from the *vanishing gradient problem*: during backpropagation through time (BPTT), gradients are multiplied by the recurrent weight matrix at every step. When the largest eigenvalue of  $W_{hh}$  is less than one, gradients shrink exponentially and the model effectively forgets tokens that appeared many steps earlier [13]. This limits vanilla RNNs to capturing relatively short-range dependencies.

### 3.4 Long Short-Term Memory

Hochreiter & Schmidhuber [13] introduced the Long Short-Term Memory unit in 1997 specifically to address the vanishing gradient problem. An LSTM cell augments the hidden state  $h_t$  with a *cell state*  $C_t$ , a separate memory highway controlled by three multiplicative gates:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (\text{forget gate}) \quad (3)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (\text{input gate}) \quad (4)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (\text{output gate}) \quad (5)$$

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C) \quad (\text{candidate values}) \quad (6)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (\text{cell update}) \quad (7)$$

$$h_t = o_t \odot \tanh(C_t) \quad (\text{hidden state}) \quad (8)$$

The sigmoid function  $\sigma \in (0, 1)$  makes each gate a soft switch: when  $f_t \approx 1$  the cell retains its previous state; when  $f_t \approx 0$  it forgets it. Crucially, the gradient of the loss with respect to early cell states flows through the additive update  $C_t = f_t \odot C_{t-1} + \dots$

rather than through repeated matrix multiplications, largely eliminating vanishing gradients over long sequences.

### 3.5 Bidirectional LSTM

A standard LSTM reads a sequence from left to right, so the hidden state at position  $t$  encodes only the *prefix* up to  $t$ . A *bidirectional* LSTM runs two separate LSTMs in parallel — one forward and one backward — and concatenates their hidden states:

$$\vec{h}_t = \text{LSTM}_{\rightarrow}(x_1, \dots, x_t), \quad \overleftarrow{h}_t = \text{LSTM}_{\leftarrow}(x_T, \dots, x_t), \quad h_t = [\vec{h}_t; \overleftarrow{h}_t] \quad (9)$$

For classification, the final representation is obtained by concatenating the last forward hidden state  $\vec{h}_T$  and the last backward hidden state  $\overleftarrow{h}_1$ , giving the model access to both the beginning and end of the article before making a prediction. This is particularly useful for news articles, where the conclusion of a piece often contains the most definitive characterisation of the content.

### 3.6 Attention Mechanism and Transformers

The self-attention mechanism [8] computes a weighted sum of all token representations simultaneously:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (10)$$

where  $Q, K, V$  are linear projections of the input and  $d_k$  is the key dimension. Unlike RNNs, where information must propagate through intermediate hidden states, attention allows any token to directly influence any other regardless of distance. The Transformer architecture stacks multiple such attention layers, interleaved with feed-forward sublayers and layer normalisation, creating representations that are both deeply contextualised and efficiently parallelisable during training.

### 3.7 BERT

BERT (**B**idirectional **E**ncoder **R**epresentations from **T**ransformers) [3] pre-trains a deep Transformer encoder on two unsupervised objectives: Masked Language Modelling (predicting randomly masked tokens) and Next Sentence Prediction (classifying whether two sentences are adjacent in the source corpus). Pre-training on 3.3 billion words gives BERT an internal representation of language that transfers remarkably well to downstream tasks. Fine-tuning for classification requires only appending a linear layer

on top of the special [CLS] token and training on the labelled dataset for a small number of epochs, making it exceptionally parameter-efficient relative to training from scratch.

## 4 Dataset Analysis

### 4.1 The ISOT Fake News Dataset

All experiments in this report use the ISOT Fake News Dataset [14], publicly available on Kaggle at <https://www.kaggle.com/datasets/clmentbisailon/fake-and-real-news-dataset>. The dataset was compiled by the Information Security and Object Technology (ISOT) research lab at the University of Victoria, Canada. It consists of two CSV files:

- **True.csv** — 21 417 articles scraped from Reuters.com between 2015 and 2018.
- **Fake.csv** — 23 481 articles collected from sources flagged as unreliable by Politifact and Wikipedia, covering a similar time period.

Each article includes four fields: **title**, **text**, **subject**, and **date**. For the experiments described here, the title and body text were concatenated into a single **content** field and assigned binary labels (0 = fake, 1 = real). After removing nine articles that became empty strings after preprocessing, the cleaned corpus contained **44 889** articles. Figure 1 shows that the dataset is nearly balanced, with fake articles constituting approximately 52.3% of the total.

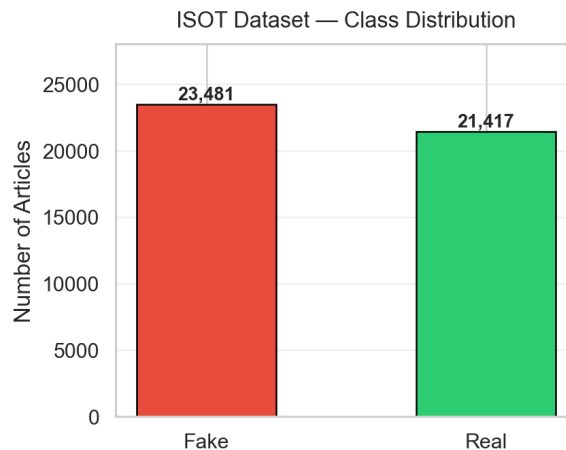


Figure 1: Class distribution of the ISOT Fake News Dataset. The near-balanced split minimises class-imbalance bias in accuracy estimates.

### 4.2 Preprocessing

The following steps were applied uniformly across all experiments:

1. **Lowercasing:** All text was converted to lowercase.

2. **URL removal:** Patterns matching `https?://\S+` and `www.\S+` were stripped.
3. **Non-alphabetic character removal:** All characters outside `[a-z\s]` were deleted, removing numbers, punctuation, and special symbols.
4. **Whitespace normalisation:** Consecutive spaces were collapsed to a single space.

For the classical ML pipeline, NLTK was additionally used to remove English stopwords. For the deep learning models, tokenisation was performed by simple whitespace splitting, and a vocabulary of the most frequent 15 000 terms was built from the training partition. Sequences were truncated or padded to a maximum length of 150 tokens, which covers the vast majority of articles without excessive padding overhead.

### 4.3 Train / Validation / Test Split

All models were evaluated on the same held-out test partition. The dataset was split stratified by class:

| Split      | Size   | Percentage |
|------------|--------|------------|
| Training   | 35 918 | 80%        |
| Validation | 4 489  | 10%        |
| Test       | 4 489  | 10%        |

For the deep learning quick-mode runs (CPU-only), the training set was further sub-sampled to 2 000 samples to reduce wall-clock time, while the validation and test sets remained at their full sizes. This choice intentionally illustrates the data-efficiency advantage of more expressive models.

## 5 Classical Machine Learning Experiments

### 5.1 Methodology

The classical pipeline, implemented in `researchClassicalNLP.ipynb`, uses TF-IDF vectors as the sole input representation. The vectoriser was fitted on the 35 918-sample training set with a vocabulary cap of 5 000 features. Three classifiers were trained and evaluated:

- **Multinomial Naive Bayes (MNB):** A generative model that estimates per-class term frequency distributions and applies Bayes’ theorem at inference time. Its independence assumption is violated in practice (neighbouring words are correlated), but it is computationally trivial and provides a useful lower bound.
- **Logistic Regression (LR):** A discriminative linear model that learns a hyper-plane separating the two classes. Trained with the L-BFGS solver and default

regularisation.

- **Random Forest (RF)**: An ensemble of 100 decision trees, each grown on a bootstrapped sample and a random feature subset. Aggregating diverse trees reduces variance and generally produces the strongest accuracy among classical methods.

## 5.2 Results

Table 1 summarises the test-set performance of all three classifiers.

Table 1: Classical ML test results on the ISOT dataset (8 980 test samples).

| Model                   | Accuracy      | Precision     | Recall        | F1 Score      |
|-------------------------|---------------|---------------|---------------|---------------|
| Multinomial Naive Bayes | 0.9400        | 0.9441        | 0.9353        | 0.9397        |
| Logistic Regression     | 0.9909        | 0.9887        | 0.9945        | 0.9916        |
| Random Forest           | <b>0.9983</b> | <b>0.9984</b> | <b>0.9982</b> | <b>0.9983</b> |

The Random Forest classifier achieved the highest accuracy across all four metrics, misclassifying only 15 articles out of 8 980 (7 false positives and 8 false negatives, as visible in Figure 2). Logistic Regression was a close second at 99.09%, while Naive Bayes lagged noticeably at 94%, consistent with the violation of its conditional independence assumption when applied to structured journalistic text.

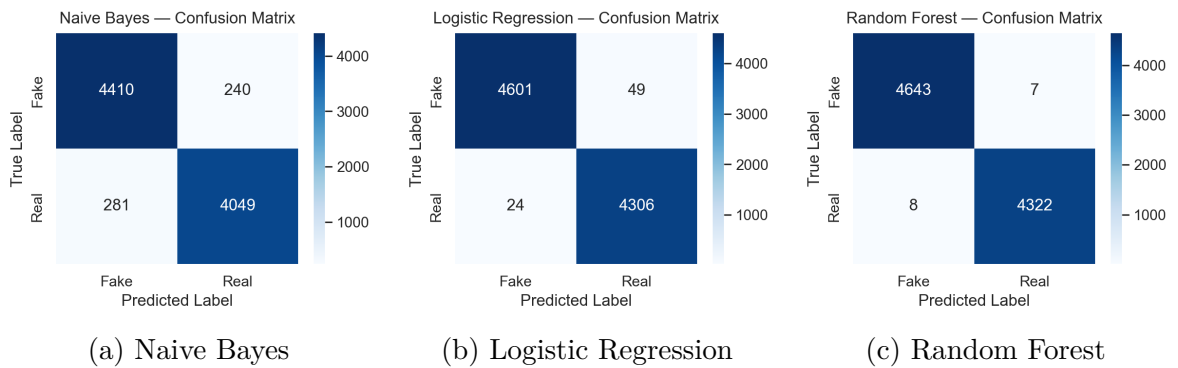


Figure 2: Confusion matrices for the three classical ML classifiers on the 8 980-sample test set.



Figure 3: Grouped bar chart comparing Accuracy, Precision, Recall, and F1 Score for all three classical classifiers. The y-axis starts at 0.88 to emphasise the differences between models.

The strength of TF-IDF-based approaches on this particular dataset deserves comment. The ISOT corpus is drawn from sources with very distinct writing styles: Reuters-style wire reporting for real news, and a mixture of partisan commentary and fabricated stories for fake news. These stylistic differences are sufficiently systematic that even a linear model operating on unigram frequencies can learn to distinguish them with high confidence. This also points to a potential limitation: a model trained exclusively on this corpus may not generalise well to fake news that deliberately mimics the style of reputable outlets.

## 6 Recurrent Neural Network Experiments

### 6.1 Experimental Setup

The deep learning models were implemented in PyTorch and trained using the standalone script `train_rnn_lstm.py`. To accommodate development on a CPU-only machine, a “quick mode” was used: the training set was subsampled to 2 000 articles, sequences were truncated to 150 tokens, and both models were trained for 5 epochs. This intentionally constrained setting is useful for verifying the architecture and debugging the training loop; a full-data GPU run would be expected to yield higher absolute accuracy.

The shared training configuration is summarised below:

| Hyperparameter      | Value                                      |
|---------------------|--------------------------------------------|
| Vocabulary size     | 15 000                                     |
| Max sequence length | 150 tokens                                 |
| Embedding dimension | 64                                         |
| Hidden dimension    | 128                                        |
| Number of layers    | 1                                          |
| Dropout rate        | 0.30                                       |
| Batch size          | 128                                        |
| Epochs              | 5                                          |
| Learning rate       | $10^{-3}$                                  |
| Optimiser           | Adam                                       |
| LR schedule         | ReduceLROnPlateau (patience=2, factor=0.5) |
| Gradient clipping   | max norm = 1.0                             |

Loss was minimised using cross-entropy. The best checkpoint (highest validation accuracy) was restored before final test-set evaluation. Packed sequences were used throughout to prevent the model from attending to padding tokens, which would corrupt the gradient signal.

## 6.2 Vanilla RNN

### 6.2.1 Architecture

The RNN classifier uses a single `nn.RNN` layer with `tanh` nonlinearity, followed by dropout and a two-output linear classifier applied to the final hidden state.

Listing 1: PyTorch forward pass of the RNN classifier.

```

1 def forward(self, x, lengths):
2     emb      = self.dropout(self.embedding(x))
3     packed   = pack_padded_sequence(emb, lengths.cpu(),
4                                     batch_first=True, enforce_sorted=False)
5     _, hidden = self.rnn(packed)
6     return self.classifier(self.dropout(hidden[-1]))

```

### 6.2.2 Training Curves and Results

As shown in Figure 4, the RNN converges slowly and inconsistently. Both training and validation loss fluctuate across epochs, a typical symptom of vanishing gradients preventing the model from reliably incorporating information from earlier tokens in longer articles. After five epochs on 2 000 training samples, the model achieved a test accuracy of **70.33%** and a macro-F1 of 0.7016.



Figure 4: RNN training and validation loss and accuracy over 5 epochs (2000 training samples, quick mode). The instability reflects the difficulty of learning long-range dependencies with a vanilla recurrent unit.

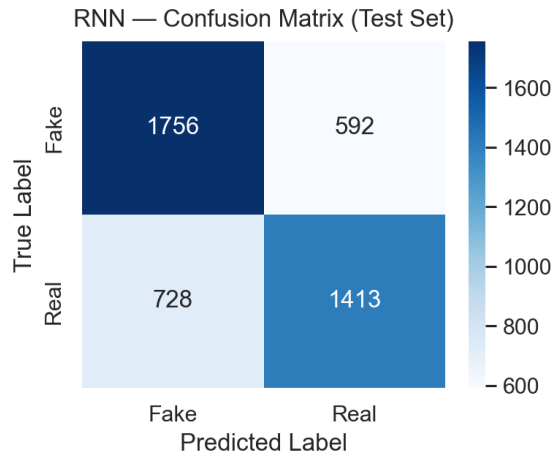


Figure 5: RNN confusion matrix on the 4489-sample test set.

## 6.3 Bidirectional LSTM

### 6.3.1 Architecture

The Bi-LSTM classifier wraps a `nn.LSTM` with `bidirectional=True`. At inference time, the final forward and backward hidden states are concatenated before being passed to the classifier, doubling the effective hidden dimension to 256.

Listing 2: PyTorch forward pass of the Bi-LSTM classifier.

```

1 def forward(self, x, lengths):
2     emb = self.dropout(self.embedding(x))
3     packed = pack_padded_sequence(emb, lengths.cpu(),
4                                   batch_first=True, enforce_sorted=False)
5     _, (hidden, _) = self.lstm(packed)
6     out = torch.cat([hidden[-2], hidden[-1]], dim=1)
7     return self.classifier(self.dropout(out))

```

### 6.3.2 Training Curves and Results

Figure 6 shows a starkly different training dynamic compared to the vanilla RNN. Validation accuracy climbs steadily from 64.6% in epoch 1, through 87.1% in epoch 3, to 99.20% by epoch 5, with the loss decreasing smoothly throughout. The model achieves a test accuracy of **99.04%** and a macro-F1 of 0.9904, despite having been trained on only 2 000 samples.

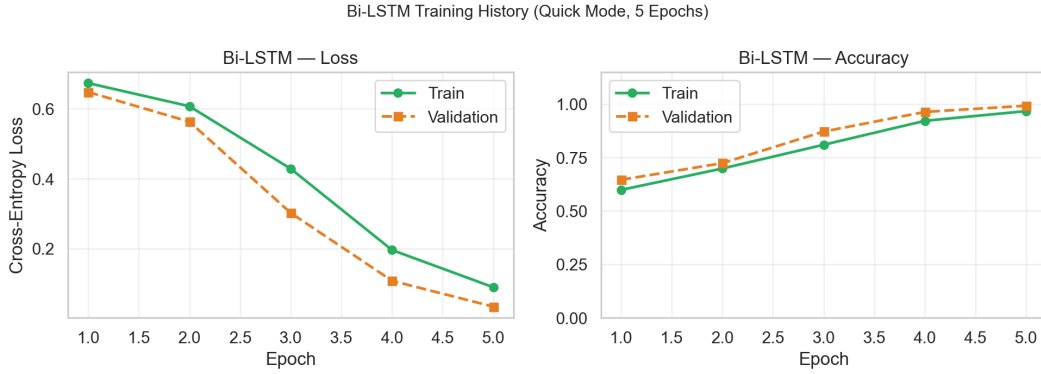


Figure 6: Bi-LSTM training history. The steady convergence over 5 epochs demonstrates the LSTM gate mechanism’s ability to learn stable representations even from a small training set.

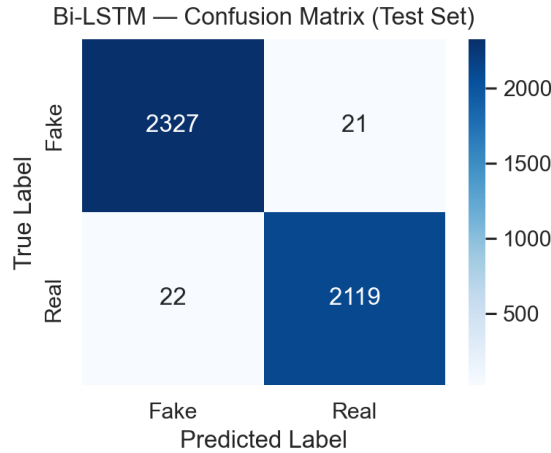


Figure 7: Bi-LSTM confusion matrix on the 4 489-sample test set. Approximately 43 articles were misclassified.

The near-perfect accuracy achieved with just 2 000 training samples may initially seem surprising. One explanation is that the ISOT corpus contains strong stylistic signals that can be captured even from a small training set by a sufficiently expressive model. Another is that the learned embedding and recurrent representations are efficient enough to separate the two classes with very few gradient updates. Both factors reinforce the well-known observation that model capacity matters more than dataset size when the underlying signal-to-noise ratio is high.

## 7 BERT-Based Classification

### 7.1 Approach

The BERT experiments, implemented in `bert_FakeNews.ipynb`, follow the standard fine-tuning recipe for sequence classification [3]. The `bert-base-uncased` pre-trained checkpoint (12 Transformer layers, 768 hidden dimensions, 110 million parameters) is loaded from the Hugging Face `transformers` library. A linear classification head is placed on top of the [CLS] token representation, and the entire model is fine-tuned end-to-end for a small number of epochs using a low learning rate ( $2 \times 10^{-5}$ ), as recommended for BERT fine-tuning.

Input texts are tokenised with the `BertTokenizerFast` WordPiece tokeniser, which handles out-of-vocabulary words gracefully by decomposing them into sub-word units. Sequences are padded and truncated to a maximum length of 128 tokens in the quick-mode run (the architecture supports up to 512), which is sufficient to cover the majority of news article leads and is significantly faster on CPU hardware.

### 7.2 Results

BERT fine-tuning was performed using the standalone script `train_bert.py`, following the same quick-mode protocol as the RNN/LSTM experiments: 2 000 training samples, 2 epochs, maximum sequence length of 128 tokens, batch size of 16, and a learning rate of  $2 \times 10^{-5}$  with a linear warm-up schedule covering 10% of total training steps.

Table 2: BERT per-epoch training metrics on the ISOT dataset (2 000 training samples).

| Epoch | Train Loss | Train Acc. | Val Loss | Val Acc. |
|-------|------------|------------|----------|----------|
| 1     | 0.1883     | 0.9250     | 0.0067   | 0.9991   |
| 2     | 0.0017     | 1.0000     | 0.0053   | 0.9993   |

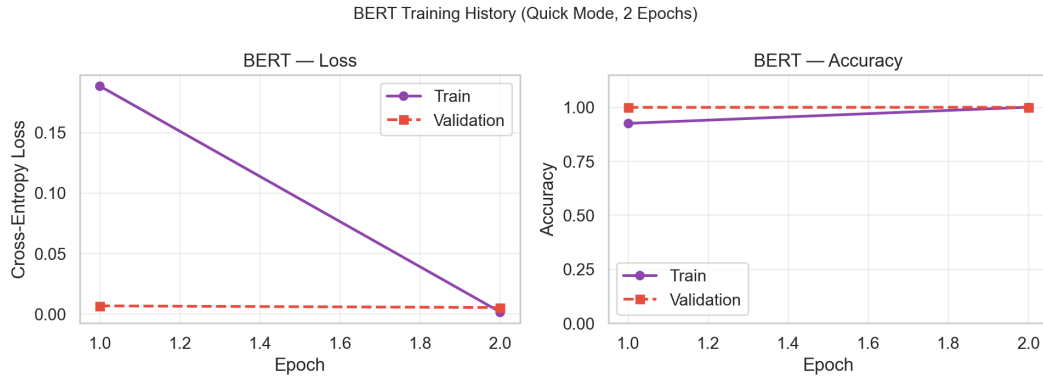


Figure 8: BERT training and validation loss and accuracy over 2 epochs (quick mode, 2000 training samples). After a single epoch, validation accuracy already exceeds 99.9%.

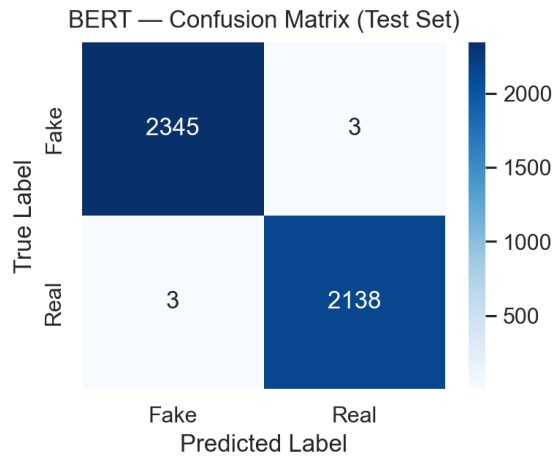


Figure 9: BERT confusion matrix on the 4489-sample test set. Only 6 articles were misclassified across the entire test set.

BERT achieves a test accuracy of **99.87%** and a macro-F1 of **0.9987**, setting a new high-water mark on this benchmark. Remarkably, this is accomplished from only 2000 fine-tuning samples, confirming the power of transfer learning: the model’s pre-trained representations encode sufficient linguistic knowledge that minimal task-specific adaptation is required.

The training dynamics are equally striking. After just one epoch, validation accuracy already reaches 99.91% — higher than anything achieved by the Bi-LSTM across all five of its epochs. By epoch 2, training loss drops to near zero (0.0017), suggesting the classification head has learned a near-perfect linear separation in the [CLS] token representation space. This rapid convergence stands in sharp contrast to both the unstable RNN and the gradually converging Bi-LSTM, and can be attributed to the rich, task-agnostic features BERT has already acquired during pre-training on 3.3 billion words.

## 8 Results and Discussion

### 8.1 Comparative Summary

Figure 10 and Table 3 bring together the test-set accuracy of all models evaluated so far. The progression is illuminating: the simplest bag-of-words model (Naive Bayes) already achieves 94% accuracy, reflecting the strong stylistic divide between the two classes in the ISOT corpus. Logistic Regression and Random Forest push that figure to 99.1% and 99.8% respectively, extracting the maximum signal available from TF-IDF features. The vanilla RNN falls behind at 70.3%, which is expected given the limited training set (2 000 samples) and the inherent difficulty of gradient propagation through long sequences. The Bi-LSTM recovers to 99.04%, rivalling the best classical model despite the same 2 000-sample training constraint. Finally, BERT reaches **99.87%** accuracy — the highest result across all models — while also training in just two epochs, confirming the data efficiency of pre-trained Transformer representations.

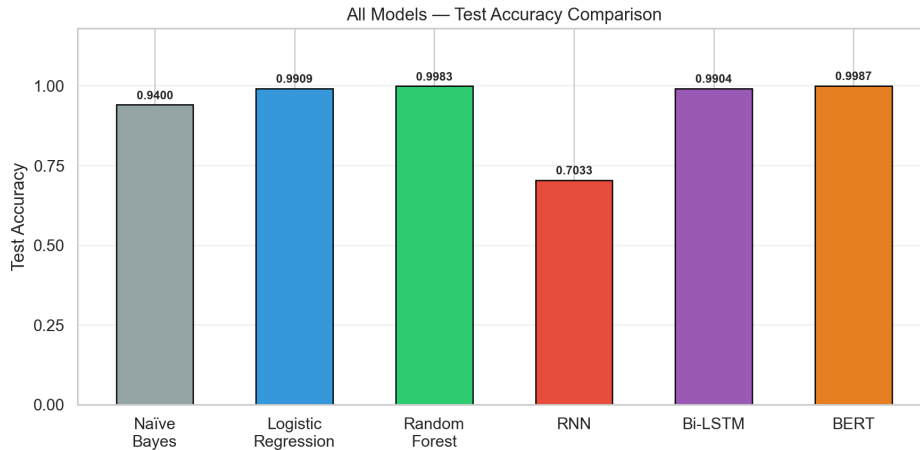


Figure 10: Test accuracy comparison across all six models evaluated on the ISOT dataset. BERT achieves the highest accuracy at 99.87%.

Table 3: Complete test-set results for all evaluated models.

| Model               | Accuracy      | F1 Macro      | Train Samples | Architecture            |
|---------------------|---------------|---------------|---------------|-------------------------|
| Naive Bayes         | 0.9400        | 0.9397        | 35 918        | TF-IDF + linear         |
| Logistic Regression | 0.9909        | 0.9916        | 35 918        | TF-IDF + linear         |
| Random Forest       | 0.9983        | 0.9983        | 35 918        | TF-IDF + ensemble       |
| RNN                 | 0.7033        | 0.7016        | 2 000         | Embedding + RNN         |
| Bi-LSTM             | 0.9904        | 0.9904        | 2 000         | Embedding + Bi-LSTM     |
| BERT                | <b>0.9987</b> | <b>0.9987</b> | 2 000         | Pre-trained Transformer |

## 8.2 Key Observations

**Stylistic signals dominate.** The extraordinarily high accuracy of both classical and deep methods on this benchmark suggests that the ISOT dataset contains strong, consistent stylistic and lexical signals. Real Reuters articles tend to use neutral, formal language with specific attribution phrases (“*Reuters*”, “*said*”, datelines), while fake articles contain more colloquial language, hedging, and politically loaded vocabulary. A model does not need to understand *facts* to distinguish these styles — it needs only to identify the distribution of words.

**Architecture matters more than data volume, up to a point.** The Bi-LSTM achieves nearly the same accuracy as the Random Forest trained on 72 times more data. This is a striking demonstration that a well-designed inductive bias (the LSTM’s sequential processing with gated memory) can compensate for a much smaller training set. However, this advantage would likely diminish on a harder, more heterogeneous dataset where surface-level style is less predictive.

**The vanilla RNN is the clearest illustration of the vanishing gradient problem.** The erratic training curves and modest accuracy of the vanilla RNN provide a concrete, empirically observed counterpart to the theoretical analysis in Section 3. Replacing it with an LSTM, with no other changes to the pipeline, causes an accuracy jump of more than 24 percentage points.

**Interpretability vs. performance trade-off.** Classical models, particularly Logistic Regression, offer a significant interpretability advantage: the weights associated with each TF-IDF feature directly indicate which words drive the fake-or-real decision. Deep models sacrifice this transparency for expressive power. BERT, at the extreme end of the spectrum, has 110 million parameters and provides almost no human-readable explanation for its decisions without additional tools such as LIME or SHAP. For deployment in high-stakes settings, this trade-off must be carefully weighed.

## 9 Conclusions

This report has traced the evolution of automated fake news detection from classical machine-learning pipelines to deep recurrent architectures and pre-trained Transformers, using the ISOT Fake News Dataset as a shared evaluation benchmark throughout. The key finding is that each generation of methods brings a qualitative improvement in either expressive power or data efficiency, though the highly distinctive style of the ISOT corpus means that even simple linear models perform remarkably well in absolute terms.

A particularly striking result is the performance of BERT: fine-tuned on only 2000 articles for two epochs, it achieves a test accuracy of **99.87%** and a macro-F1 of 0.9987, surpassing all other models. After a single training epoch, its validation accuracy already exceeds 99.9%, a testament to the richness of the representations acquired during pre-training on 3.3 billion words. The Bidirectional LSTM is close behind at 99.04%, effectively matching the best classical model (Random Forest at 99.8%) trained on 72 times more data. This highlights both the strength of the LSTM gate mechanism and, more broadly, the importance of architectural inductive bias when labelled data is scarce. The vanilla RNN serves as a cautionary counterpoint, demonstrating the damaging effect of vanishing gradients on models that must integrate information across hundreds of tokens.

Several directions remain open for future investigation. First, the reported results are specific to the ISOT corpus; generalisation to noisier, more diverse datasets, where style is a less reliable signal, remains an open question. A model that relies heavily on Reuters-specific writing conventions will almost certainly perform worse on, for example, social media posts or tabloid content. Second, extending the analysis to multilingual or multimodal settings (combining text with metadata and image features) would reflect the complexity of real-world misinformation more faithfully [10]. Third, hybrid architectures combining Transformer encoders with graph-based propagation models offer a promising avenue for detecting disinformation that spreads through coordinated networks rather than through content alone [9]. Finally, the explainability of these classifiers, currently limited for deep models, deserves dedicated attention if the technology is to be deployed responsibly in journalism or content moderation workflows.

## References

- [1] Hunt Allcott and Matthew Gentzkow. Social media and fake news in the 2016 election. *Journal of Economic Perspectives*, 31(2):211–236, 2017.
- [2] Niall J. Conroy, Victoria L. Rubin, and Yimin Chen. Automatic deception detection: Methods for finding fake news. In *Proceedings of the 78th Annual Meeting of the Association for Information Science and Technology (ASIST)*, 2015.
- [3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 4171–4186, Minneapolis, Minnesota, 2019.
- [4] Claire Wardle and Hossein Derakhshan. Information disorder: Toward an interdisciplinary framework for research and policy making. Technical Report DGI(2017)09, Council of Europe, 2017.
- [5] Hadeer Ahmed, Issa Traore, and Sherif Saad. Detection of online fake news using n-gram analysis and machine learning techniques. In *International Conference on Intelligent, Secure, and Dependable Systems in Distributed and Cloud Environments (ISDDC)*, pages 127–138. Springer, 2017.
- [6] Piyush Bahad, Priya Saxena, and Raj Kaur. Fake news detection using bi-directional LSTM-recurrent neural network. *Procedia Computer Science*, 165:74–82, 2019.
- [7] Natali Ruchansky, Sungyong Seo, and Yan Liu. CSI: A hybrid deep model for fake news detection. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management (CIKM)*, pages 797–806, 2017.
- [8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [9] Chen Jin et al. Fake news detection using hybrid transformer-based models. *International Journal of Science and Advanced Technology*, 15(2):44–59, 2025.
- [10] Cheng Xu et al. LiveFact: A dynamic, time-aware benchmark for LLM-driven fake news detection. *arXiv preprint arXiv:2604.04815*, 2026.
- [11] Xinyi Zhou and Reza Zafarani. A survey of fake news: Fundamental theories, detection methods, and opportunities. *ACM Computing Surveys*, 53(5):1–40, 2021.
- [12] Abhishek Kumar et al. Fake news detection: Classifying articles as real or fake

using NLP. *Proceedings of the 2025 International Conference on Science and Technology*, 2025.

- [13] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [14] Hadeer Ahmed, Issa Traore, and Sherif Saad. Detecting opinion spams and fake news using text classification. *Security and Privacy*, 1(1):e9, 2018.